WordPress su Architettura a Due Macchine

Guida pratica — Linux Server

ITS Academy

Obiettivo dell'esercitazione

- Mettere in rete due macchine Debian in VirtualBox
- debian1 → frontend: nginx + WordPress (accessibile dall'esterno)
- debian2 → backend: MariaDB (accessibile SOLO da debian1 via rete interna)
- Accesso SSH a debian2 solo tramite debian1 (ProxyJump)
- Scenario realistico: separazione frontend/backend come in produzione

1

Architettura e configurazione VirtualBox

Configurazione schede di rete (NIC)

Macchina	NIC 1	NIC 2	NIC 3
debian1	NAT	Solo Host → 192.168.56.110	Rete interna intnet → 10.0.0.1
debian2	NAT	Rete interna intnet → 10.0.0.2	—

Clone VM e cambio hostname

- Clonare debian1 in VirtualBox per ottenere debian2
- Durante il clone: rigenerare i MAC address (o farlo manualmente dopo)
- Cambiare hostname su debian2:
- → echo "debian2" | sudo tee /etc/hostname
- → sudo sed -i 's/debian1/debian2/' /etc/hosts
- → sudo hostnamectl set-hostname debian2

⚠️ *Il clone condivide il MAC address originale: rigenerarlo per evitare conflitti DHCP*

2

Configurazione rete interna (intnet)

Interfacce di rete rilevate

Macchina	Interfaccia	Ruolo
debian1	enp0s3	NAT
debian1	enp0s8	Solo Host
debian1	enp0s9	Rete interna intnet
debian2	enp0s3	NAT
debian2	enp0s8	Rete interna intnet

Configurazione /etc/network/interfaces

```
# --- debian1: aggiungere in fondo ---
# Internal network interface (intnet)
# Static IP for private communication with debian2
auto enp0s9
iface enp0s9 inet static
    address 10.0.0.1
    netmask 255.255.255.0

# --- debian2: aggiungere in fondo ---
# Internal network interface (intnet)
# Static IP for private communication with debian1
auto enp0s8
iface enp0s8 inet static
    address 10.0.0.2
    netmask 255.255.255.0
```

Attivare le interfacce e testare

```
# Attivare su debian1 (usare SEMPRE sudo)
sudo ip addr add 10.0.0.1/24 dev enp0s9

# Attivare su debian2
sudo ip addr add 10.0.0.2/24 dev enp0s8

# Verificare la connettività (da debian1)
ping -c 4 10.0.0.2

# Output atteso: 0% packet loss
```

3

Configurazione MariaDB su debian2

Installazione e sicurezza di base

```
# Installazione
sudo apt update && sudo apt install mariadb-server -y

# Accesso
sudo mysql

-- Remove anonymous users
DELETE FROM mysql.user WHERE User='';
-- Disallow root login remotely
DELETE FROM mysql.user WHERE User='root'
    AND Host NOT IN ('localhost', '127.0.0.1', '::1');
DROP DATABASE IF EXISTS test;
FLUSH PRIVILEGES;
```

Creazione database e utente WordPress

```
-- Create the WordPress database
CREATE DATABASE wordpress
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Create user restricted to debian1's IP only
CREATE USER 'wpuser'@'10.0.0.1' IDENTIFIED BY 'its2026';

-- Grant privileges only on wordpress DB
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'10.0.0.1';

FLUSH PRIVILEGES;
```

Configurazione bind-address in 50-server.cnf

```
# File: /etc/mysql/mariadb.conf.d/50-server.cnf

# bind-address: IP on which MariaDB listens
# This is debian2's IP on the internal network
# NOTE: NOT the client IP — that's controlled by user privileges
bind-address = 10.0.0.2

# Riavviare il servizio
sudo systemctl restart mariadb

# Verificare
sudo ss -tlnp | grep 3306
# Output atteso: LISTEN ... 10.0.0.2:3306
```

4

Installazione WordPress su debian1

Installazione nginx e PHP-FPM

```
# Installare nginx e PHP (NON usare 'php' generico:
# trascina Apache2 che va in conflitto con nginx)
sudo apt install nginx php8.4-fpm php8.4-mysql \
    php8.4-curl php8.4-gd php8.4-mbstring \
    php8.4-xml php8.4-xmlrpc php8.4-zip -y

# Verificare
sudo systemctl status nginx
sudo systemctl status php8.4-fpm
```

Download e installazione WordPress

```
# Download
cd /tmp && sudo wget https://wordpress.org/latest.tar.gz
sudo tar -xzf /tmp/latest.tar.gz -C /tmp
sudo mv /tmp/wordpress /var/www/wordpress

# Permessi
sudo chown -R www-data:www-data /var/www/wordpress
sudo find /var/www/wordpress -type d -exec chmod 755 {} \;
sudo find /var/www/wordpress -type f -exec chmod 644 {} \;

# Aggiungere a /etc/hosts su debian1:
# 127.0.0.1   wordpress
```

Configurazione nginx — sites-available/wordpress

```
server {
    listen 80;
    server_name wordpress;
    root /var/www/wordpress;
    index index.php index.html;
    access_log /var/log/nginx/wordpress_access.log;
    error_log  /var/log/nginx/wordpress_error.log;
    location / {
        try_files $uri $uri/ /index.php?$args;
    }
    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.4-fpm.sock;
    }
    location ~ /\. { deny all; }
}
```

Configurazione wp-config.php

```
sudo cp /var/www/wordpress/wp-config-sample.php \
        /var/www/wordpress/wp-config.php

# Modificare questi parametri:
define( 'DB_NAME',     'wordpress' );
define( 'DB_USER',     'wpuser' );
define( 'DB_PASSWORD', 'its2026' );
# DB_HOST = IP interno di debian2
define( 'DB_HOST',     '10.0.0.2' );
```

5

Accesso SSH tramite ProxyJump

SSH ProxyJump — accedere a debian2 dall'esterno

```
# Comando diretto
ssh -J its-usr@192.168.56.110 its-usr@10.0.0.2

# Oppure configurare ~/.ssh/config sul proprio PC:
Host debian1
    HostName 192.168.56.110
    User its-usr

Host debian2
    HostName 10.0.0.2
    User its-usr
    ProxyJump debian1

# Da quel momento basta:
ssh debian2
```

6

Firewall su debian1 (ufw)

Configurazione firewall con ufw

```
# Installazione
sudo apt install ufw -y

# Politica default: blocca tutto in entrata, permetti in uscita
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Permetti SSH (porta 22)
sudo ufw allow 22/tcp

# Permetti HTTP (porta 80)
sudo ufw allow 80/tcp

# Attivare e verificare
sudo ufw enable
sudo ufw status verbose
```

Traffico permesso dopo il firewall

Connessione	Direzione	Permessa?
SSH dall'esterno → debian1 (porta 22)	IN	Sì
HTTP → WordPress (porta 80)	IN	Sì
debian1 → MariaDB su debian2 (3306)	OUT	Sì
debian1 → SSH su debian2 (22)	OUT	Sì
Qualsiasi altra connessione in entrata	IN	No — bloccata

ProxyJump vs Tunnel SSH — differenza

- ProxyJump: SSH si connette a debian1 e apre una NUOVA connessione SSH verso debian2
- Il traffico è cifrato end-to-end ma NON è un tunnel
- Tunnel SSH (port forwarding): fa forwarding di una porta remota sul PC locale
- → es: ssh -L 3306:10.0.0.2:3306 its-usr@192.168.56.110
- → utile per connettersi al DB con un client grafico dal proprio PC
- Per amministrare debian2 → usare ProxyJump
- Per accedere al DB da client grafico → usare tunnel SSH

Riepilogo architettura finale

Componente	Macchina	Dettaglio
nginx + WordPress	debian1	192.168.56.110 (Solo Host) / 10.0.0.1 (intnet)
PHP-FPM 8.4	debian1	Gestione script PHP
MariaDB	debian2	10.0.0.2 — non raggiungibile dall'esterno
Utente DB	debian2	wpuser@10.0.0.1 (solo da debian1)
SSH esterno	debian1	Porta 22, IP 192.168.56.110
SSH debian2	via debian1	ProxyJump: ssh debian2